

1) PRIMARY KEY add karna

SQL

```
ALTER TABLE student  
ADD PRIMARY KEY (id);
```

2) COMPOSITE PRIMARY KEY add karna

Agar 2 columns milke primary key banani ho:

SQL

```
ALTER TABLE enrollment  
ADD PRIMARY KEY (student_id, course_id);
```

3) UNIQUE KEY add karna

SQL

```
ALTER TABLE student  
ADD UNIQUE (email);
```

Agar constraint name bhi dena ho:

SQL

```
ALTER TABLE student  
ADD CONSTRAINT uq_email UNIQUE (email);
```

4) FOREIGN KEY add karna

Example: `student.dept_id` ka relation `department.id` se hai

SQL

```
ALTER TABLE student
ADD CONSTRAINT fk_dept
FOREIGN KEY (dept_id)
REFERENCES department(id);
```

5) COMPOSITE UNIQUE KEY add karna

SQL

```
ALTER TABLE employee
ADD CONSTRAINT uq_emp UNIQUE (name, phone);
```

1. Add Column

SQL

```
ALTER TABLE student  
ADD email VARCHAR(100);
```

2. Drop Column

SQL

```
ALTER TABLE student  
DROP COLUMN email;
```

3. Rename Table

⟨⟩ SQL

```
RENAME TABLE student TO students;
```

-- OR

```
ALTER TABLE students  
RENAME TO student;
```

4. Rename Column (MySQL 8.0+)

⟨⟩ SQL

```
ALTER TABLE student  
RENAME COLUMN phone TO mobile;
```

5. Change Column (Rename + Datatype)

SQL

```
ALTER TABLE student  
CHANGE COLUMN mobile phone VARCHAR(15);
```

Old Name → mobile

New Name → phone

6. Modify Column (Datatype Only)

SQL

```
ALTER TABLE student  
MODIFY COLUMN phone VARCHAR(20);
```



7. Change Column Position

First Position

SQL

```
ALTER TABLE student  
MODIFY COLUMN phone VARCHAR(15) FIRST;
```

After Another Column

SQL

```
ALTER TABLE student  
MODIFY COLUMN phone VARCHAR(15) AFTER name;
```

Solution 1 (Recommended)

SQL

```
TRUNCATE TABLE student;  
TRUNCATE TABLE department;
```

Solution 2 (Disable FK Checks)

SQL

```
SET FOREIGN_KEY_CHECKS = 0;  
  
TRUNCATE TABLE department;  
  
SET FOREIGN_KEY_CHECKS = 1;
```


Correct Order

SQL

```
DROP TABLE student;
```

```
DROP TABLE department;
```

OR

SQL

```
SET FOREIGN_KEY_CHECKS = 0;
```

```
DROP TABLE department;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

Operation	Query
Add Column	<code>ALTER TABLE student ADD email VARCHAR(100);</code>
Drop Column	<code>ALTER TABLE student DROP COLUMN email;</code>
Rename Table	<code>RENAME TABLE student TO students;</code>
Rename Column	<code>ALTER TABLE student RENAME COLUMN phone TO mobile;</code>
Change Column	<code>ALTER TABLE student CHANGE mobile phone VARCHAR(15);</code>
Modify Column	<code>ALTER TABLE student MODIFY phone VARCHAR(20);</code>
First Position	<code>ALTER TABLE student MODIFY phone VARCHAR(15) FIRST;</code>
After Column	<code>ALTER TABLE student MODIFY phone VARCHAR(15) AFTER name;</code>
✓ After Column	<code>ALTER TABLE student MODIFY phone VARCHAR(15) AFTER name;</code>
Truncate Child	<code>TRUNCATE TABLE student;</code>
Truncate Parent	<code>TRUNCATE TABLE department;</code> <i>(fails if referenced by FK)</i>
Delete Child	<code>DELETE FROM student;</code>
Delete Parent	<code>DELETE FROM department;</code> <i>(works with ON DELETE CASCADE, otherwise fails)</i>
Drop Child	<code>DROP TABLE student;</code>
Drop Parent	<code>DROP TABLE department;</code> <i>(fails if referenced by FK unless child removed or FK checks disabled)</i>

4) Undo / Rollback

Ye depend karta hai kis type ka command chalaya:

A) INSERT / UPDATE / DELETE ka undo

Agar transaction use ki hai aur COMMIT nahi kiya:

</> SQL

```
START TRANSACTION;
```

```
UPDATE student  
SET name = 'Rahul'  
WHERE id = 1;
```

```
ROLLBACK;
```

B) CREATE / DROP / ALTER / RENAME ka undo?

Normally **DDL commands** (CREATE , DROP , ALTER , RENAME) MySQL me **auto-commit** kar dete hain.

Matlab:

- RENAME TABLE → rollback nahi hota
- ALTER TABLE → rollback nahi hota
- DROP TABLE → rollback nahi hota
- DROP DATABASE → rollback nahi hota

Savepoint transaction ke andar ek **checkpoint** hota hai.

Iska use tab hota hai jab tum poori transaction rollback nahi karna chahte, **sirf ek specific point tak undo** karna chahte ho.

Simple meaning

Maan lo tumne transaction me 3 queries chalayi:

1. pehla update sahi hai
2. dusra update sahi hai
3. teesra galat ho gaya

Ab agar tum **ROLLBACK** karoge to **sab kuch undo** ho jayega.

Lekin agar tumne बीच में **SAVEPOINT** banaya tha, to tum **sirf us savepoint tak rollback** kar sakte ho.



</> SQL

START TRANSACTION;

UPDATE student
SET age = 21
WHERE id = 1;

SAVEPOINT sp1;

UPDATE student
SET age = 22
WHERE id = 2;

UPDATE student
SET age = 23
WHERE id = 3;

ROLLBACK TO sp1;

COMMIT;

Savepoint vs Rollback

ROLLBACK

poori transaction undo

`</> SQL`

```
START TRANSACTION;
```

```
UPDATE student SET age=20 WHERE id=1;
```

```
UPDATE student SET age=25 WHERE id=2;
```

```
ROLLBACK;
```

👉 dono updates undo

ROLLBACK TO savepoint

sirf savepoint ke baad wale changes undo

</> SQL

```
START TRANSACTION;
```

```
UPDATE student SET age=20 WHERE id=1;
```

```
SAVEPOINT sp1;
```

```
UPDATE student SET age=25 WHERE id=2;
```

```
ROLLBACK TO sp1;
```

```
COMMIT;
```

👉 id=1 update rahega

👉 id=2 undo ho jayega



Insert uppercase

SQL

```
INSERT INTO student(id, name, city)
VALUES (1, UPPER('tarun'), UPPER('indore'));
```

Insert lowercase

SQL

```
INSERT INTO student(id, name, city)
VALUES (2, LOWER('RAHUL'), LOWER('BHOPAL'));
```

Sabko uppercasse

SQL

```
UPDATE student
SET name = UPPER(name),
    city = UPPER(city);
```

Sabko lowercase

SQL

```
UPDATE student
SET name = LOWER(name),
    city = LOWER(city);
```

Function	Purpose	Example
UPPER()	Uppercase	SELECT UPPER('tarun'); → TARUN
LOWER()	Lowercase	SELECT LOWER('TARUN'); → tarun
LENGTH()	String ki length	SELECT LENGTH('Tarun'); → 5
CONCAT()	Strings jodna	SELECT CONCAT('Tarun',' Patidar'); → Tarun Patidar
SUBSTRING()	String ka part nikalna	SELECT SUBSTRING('Tarun',2,3); → aru
LEFT()	Left se characters	SELECT LEFT('Tarun',3); → Tar
RIGHT()	Right se characters	SELECT RIGHT('Tarun',2); → un
TRIM()	Dono side ke spaces remove	SELECT TRIM(' Tarun '); → Tarun
LTRIM()	Left spaces remove	SELECT LTRIM(' Tarun');
RTRIM()	Right spaces remove	SELECT RTRIM('Tarun ');
REPLACE()	String replace karna	SELECT REPLACE('Tarun','Tar','Kar'); → Karun
REVERSE()	String reverse	SELECT REVERSE('Tarun'); → nuraT
INSTR()	Character ki position	SELECT INSTR('Tarun','r'); → 3
LOCATE()	Character/String ki position	SELECT LOCATE('ru','Tarun'); → 3

```
CREATE TABLE student (  
    id INT PRIMARY KEY,  
  
    phone VARCHAR(10),  
    CHECK (phone REGEXP '^[6-9][0-9]{9}$'),  
  
    aadhaar CHAR(12),  
    CHECK (aadhaar REGEXP '^[0-9]{12}$'),  
  
    dob VARCHAR(10),  
    CHECK (dob REGEXP '^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/[0-9]{4}$'),  
  
    password VARCHAR(16),  
    CHECK (  
        CHAR_LENGTH(password) = 16  
        AND password REGEXP '[A-Z]'  
        AND password REGEXP '[a-z]'  
        AND password REGEXP '[0-9]'  
        AND password REGEXP '[@#$]'  
    )  
);
```



```
CHECK (  
  CHAR_LENGTH(password)=16  
  AND password REGEXP '[A-Z]'  
  AND password REGEXP '[a-z]'  
  AND password REGEXP '[0-9]'  
  AND password REGEXP '[@#$]'  
)
```

1. Delete Row

SQL

```
DELETE FROM student  
WHERE id = 1;
```

Undo (Transaction ke andar)

SQL

```
START TRANSACTION;  
  
DELETE FROM student  
WHERE id = 1;  
  
ROLLBACK;
```

Full SQL

Delete one row

SQL

```
DELETE FROM student WHERE id = 1;
```

Delete all rows

SQL

```
DELETE FROM student;
```

Delete column

SQL

```
ALTER TABLE student DROP COLUMN phone;
```

-- Sirf ek record delete

```
DELETE FROM student WHERE id = 1;
```

-- A, B, C delete

```
DELETE FROM student WHERE name IN ('A','B','C');
```

-- B ko chhodkar sab delete

```
DELETE FROM student WHERE name <> 'B';
```

-- A, B, C ko chhodkar sab delete

```
DELETE FROM student WHERE name NOT IN ('A','B','C');
```

-- Age 18 se kam delete

```
DELETE FROM student WHERE age < 18;
```

-- NULL values delete

```
DELETE FROM student WHERE email IS NULL;
```



1. Equal (=)

SELECT

⟨/⟩ SQL

```
SELECT * FROM student  
WHERE age = 20;
```

UPDATE

⟨/⟩ SQL

```
UPDATE student  
SET city = 'Indore'  
WHERE age = 20;
```

DELETE

⟨⟩ SQL

```
DELETE FROM student  
WHERE age = 20;
```

2. Not Equal (!=)

SELECT

⟨⟩ SQL

```
SELECT * FROM student  
WHERE age != 20;
```

UPDATE

⌞⌟ SQL

```
UPDATE student  
SET city = 'Bhopal'  
WHERE age != 20;
```

DELETE

⌞⌟ SQL

```
DELETE FROM student  
WHERE age != 20;
```

3. Not Equal (<>)

SELECT

`</>` SQL

```
SELECT * FROM student  
WHERE age <> 20;
```

UPDATE

`</>` SQL

```
UPDATE student  
SET city = 'Ujjain'  
WHERE age <> 20;
```

UPDATE

⌞ SQL

```
UPDATE student  
SET city = 'Ujjain'  
WHERE age <> 20;
```

DELETE

⌞ SQL

```
DELETE FROM student  
WHERE age <> 20;
```

4. Greater Than (>)

SELECT

SQL

```
SELECT * FROM student  
WHERE age > 20;
```

UPDATE

SQL

```
UPDATE student  
SET city = 'Delhi'  
WHERE age > 20;
```

</> SQL

```
SELECT *  
FROM student  
WHERE city = 'Indore'  
AND marks > 80;
```

👉 Sirf wahi students aayenge jo **Indore** ke hain **aur** jinke marks **80** se zy

UPDATE

</> SQL

```
UPDATE student  
SET marks = marks + 5  
WHERE city = 'Indore'  
AND course_year = 1;
```

👉 Sirf **Indore** ke **1st year** students ke marks 5 se inc ↓ e honge.

SQL

```
DELETE FROM student  
WHERE city = 'Dhar'  
AND marks < 75;
```

👉 Sirf Dhar ke students jinke marks 75 se kam hain delete honge.

2. Bitwise AND (&)

Meaning: Numbers ke **binary bits** compare karta hai.

Example:

```
5 = 101  
3 = 011  
-----  
001 = 1
```



Odd numbers ka last bit **1** hota hai.

</> SQL

```
SELECT *  
FROM student  
WHERE (marks & 1) = 1;
```

👉 Jinke marks **odd** hain (69, 91, 83...) wo students aayenge.

Even marks:

</> SQL

```
SELECT *  
FROM student  
WHERE (marks & 1) = 0;
```

👉 Jinke marks **even** hain (78, 82, 84...) wo students  nge.

UPDATE

SQL

```
UPDATE student  
SET marks = marks + 2  
WHERE (marks & 1) = 1;
```

👉 Sirf odd marks wale students ke marks 2 se badhenge.

DELETE

SQL

```
DELETE FROM student  
WHERE (marks & 1) = 0;
```

👉 Sirf even marks wale students delete ho jayenge.



Ek aur Bitwise AND Example

IDs me bitwise AND:

SQL

```
SELECT id, name
FROM student
WHERE (id & 1) = 1;
```

👉 Odd IDs:

```
1
3
5
7
...
65
```

SQL

```
SELECT *  
FROM student  
WHERE city='Indore'  
OR marks>90;
```

Output:

- Indore ke saare students
- Ya jinke marks 90 se jyada hain

UPDATE

SQL

```
UPDATE student  
SET marks=100  
WHERE city='Indore'  
OR marks>90;
```

SELECT

SQL

```
SELECT *  
FROM student  
WHERE NOT city='Indore';
```

Output:

Indore ko chhodkar baaki sab.

UPDATE

SQL

```
UPDATE student  
SET city='Delhi'  
WHERE NOT city='Indore';
```

Indore ke alawa sabki city Delhi ho jayegi.

DELETE

SQL

```
DELETE FROM student  
WHERE NOT city='Indore';
```

Indore ko chhodkar sab delete.

3. NOR

SQL me **NOR** operator nahi hota.

NOR ka matlab hota hai

```
NOT(A OR B)
```

1. IN Operator

Definition

IN operator multiple values me se match karta hai.

Iska matlab:

</> SQL

```
WHERE city='Indore'  
OR city='Bhopal'  
OR city='Ujjain'
```

ko short me likhte hain:

</> SQL

```
WHERE city IN ('Indore','Bhopal','Ujjain')
```

SELECT

 SQL

```
SELECT *  
FROM student  
WHERE city IN ('Indore','Bhopal','Ujjain');
```

 Indore, Bhopal aur Ujjain ke students.

UPDATE

 SQL

```
UPDATE student  
SET marks = marks + 5  
WHERE city IN ('Indore','Bhopal');
```

 Indore aur Bhopal ke students ke marks 5 se badhayeinge. 

2. NOT IN Operator

Definition

List me jo values hain unhe chhodkar baaki sab select karega.

SELECT

</> SQL

```
SELECT *  
FROM student  
WHERE city NOT IN ('Indore','Bhopal');
```

👉 Indore aur Bhopal ko chhodkar baaki sab students.

UPDATE

SQL

```
UPDATE student  
SET marks = 50  
WHERE city NOT IN ('Indore','Bhopal');
```

👉 Indore aur Bhopal ke alawa sabke marks 50 ho jayenge.

DELETE

SQL

```
DELETE FROM student  
WHERE city NOT IN ('Indore','Bhopal');
```

👉 Sirf Indore aur Bhopal ke students bachenge.

3. BETWEEN Operator

Definition

Range ke andar ki values select karta hai.

Equivalent to:

```
</> SQL
```

```
age >=18  
AND age<=20
```

SELECT

```
</> SQL
```

```
SELECT *  
FROM student  
WHERE age BETWEEN 18 AND 20;
```

UPDATE

SQL

```
UPDATE student  
SET marks = marks + 10  
WHERE age BETWEEN 18 AND 20;
```

👉 18–20 age ke students ke marks 10 se badh jayenge.

DELETE

SQL

```
DELETE FROM student  
WHERE age BETWEEN 18 AND 20;
```



👉 18–20 age ke students delete ho jayenge.

BETWEEN with Marks

SELECT

SQL

```
SELECT *  
FROM student  
WHERE marks BETWEEN 80 AND 90;
```

👉 80 se 90 (inclusive) marks wale students.

UPDATE

SQL

```
UPDATE student  
SET city='Indore'  
WHERE marks BETWEEN 80 AND 90;
```

DELETE

`</>` SQL

```
DELETE FROM student  
WHERE marks BETWEEN 80 AND 90;
```

NOT BETWEEN

`</>` SQL

```
SELECT *  
FROM student  
WHERE age NOT BETWEEN 18 AND 20;
```

👉 18–20 ke alawa sab.

1. LIKE Operator

Definition

LIKE string (text) me **pattern search** karne ke liye use hota hai.

Wildcards:

- % → 0 ya usse zyada characters
- _ → Exactly 1 character

% (Zero or More Characters)

A se start

</> SQL

```
SELECT *  
FROM student  
WHERE name LIKE 'A%';
```

Match:

Aarav
Aman
Aditya

n se end

</> SQL

```
SELECT *  
FROM student  
WHERE name LIKE '%n';
```

Match:

Aman

"ar" contain kare

</> SQL

```
SELECT *  
FROM student  
WHERE name LIKE '%ar%';
```

Match:

Tarun

Pattern	Meaning
'A%'	Starts with A
'%A'	Ends with A
'%A%'	Contains A
'A_____'	A + total 5 chars
'_____'	Exactly 5 chars
'_a%'	Second letter a

```
</> SQL
```

```
SELECT *  
FROM student  
WHERE marks > ANY  
(  
  SELECT marks  
  FROM topper  
);
```

ANY ka matlab

List ke kisi ek value ke saath condition true honi chahiye.

Topper list

60

80

95



/04512240-4440-0000-0070-000000000000

ALL Operator

Same topper

60

80

95

Query

</> SQL

```
SELECT *  
FROM student  
WHERE marks > ALL  
(  
  SELECT marks  
  FROM topper  
);
```

LIKE

Pattern matching (%, _)

ANY

Kisi ek value ke saath condition true (OR jaisa)

ALL

Har value ke saath condition true (AND jaisa)

DCL (Data Control Language)

DCL ka use **users ko permissions (privileges) dene aur wapas lene** ke liye hota hai.

DCL ke main commands:

1. GRANT
 2. REVOKE
-

1. GRANT

Definition

Kisi user ko database ya table par **permission dena**.

SQL

```
GRANT permission  
ON database.table  
TO 'username'@'localhost';
```

A. SELECT Permission

User sirf data dekh sakta hai.

SQL

```
GRANT SELECT  
ON college.student  
TO 'tarun'@'localhost';
```

B. INSERT Permission

SQL

```
GRANT INSERT
ON college.student
TO 'tarun'@'localhost';
```

C. UPDATE Permission

SQL

```
GRANT UPDATE
ON college.student
TO 'tarun'@'localhost';
```

D. DELETE Permission

SQL

```
GRANT DELETE
ON college.student
TO 'tarun'@'localhost';
```

E. Multiple Permissions

SQL

```
GRANT SELECT, INSERT, UPDATE
ON college.student
TO 'tarun'@'localhost';
```

F. All Permissions

SQL

```
GRANT ALL PRIVILEGES  
ON college.student  
TO 'tarun'@'localhost';
```

G. Entire Database

SQL

```
GRANT ALL PRIVILEGES  
ON college.*  
TO 'tarun'@'localhost';
```


H. Entire Server

`</> SQL`

```
GRANT ALL PRIVILEGES  
ON *.*  
TO 'tarun'@'localhost';
```

Means

Sabhi databases

WITH GRANT OPTION

User dusre users ko bhi permission de sakta hai.

SQL

```
GRANT SELECT
ON college.student
TO 'tarun'@'localhost'
WITH GRANT OPTION;
```

2. REVOKE

Definition

Di hui permission wapas lena.

Syntax

SQL

```
REVOKE permission
ON database.table
FROM 'username'@'localhost';
```

A. Remove SELECT

SQL

```
REVOKE SELECT
ON college.student
FROM 'tarun'@'localhost';
```

B. Remove INSERT

SQL

```
REVOKE INSERT
ON college.student
FROM 'tarun'@'localhost';
```

C. Remove UPDATE

⌕ SQL

```
REVOKE UPDATE  
ON college.student  
FROM 'tarun'@'localhost';
```

D. Remove DELETE

⌕ SQL

```
REVOKE DELETE  
ON college.student  
FROM 'tarun'@'localhost';
```

E. Remove Multiple Permissions

SQL

```
REVOKE SELECT, INSERT
ON college.student
FROM 'tarun'@'localhost';
```

F. Remove All Permissions

SQL

```
REVOKE ALL PRIVILEGES
ON college.student
FROM 'tarun'@'localhost';
```

MySQL 8.0 me pehle user create karna padta hai, phir us user ko permissions di jaa

SQL

```
CREATE USER 'tarun'@'localhost'
IDENTIFIED BY 'Tarun@123';

GRANT SELECT, INSERT
ON college.student
TO 'tarun'@'localhost';
```

TCL (Transaction Control Language)

TCL ka use transaction ko **start**, **save**, **commit** ya **rollback** karne ke liye hota hai

TCL Commands

1. `START TRANSACTION` (ya `BEGIN`)
2. `COMMIT`
3. `ROLLBACK`
4. `SAVEPOINT`
5. `ROLLBACK TO SAVEPOINT`
6. `RELEASE SAVEPOINT`
7. `SET AUTOCOMMIT`

Transaction Kya Hai?

Transaction = SQL statements ka ek group jo **ek unit** ki tarah execute hota hai.

Example:

ATM Transfer

Account A (-1000)

↓

Account B (+1000)

Dono statements successful honi chahiye.

Agar ek fail ho jaye to dono cancel.

Example

SQL

```
START TRANSACTION;
```

```
UPDATE student  
SET marks=95  
WHERE id=1;
```

```
UPDATE student  
SET marks=90  
WHERE id=2;
```

3. ROLLBACK

Saare changes undo.

</> SQL

```
ROLLBACK;
```

Example

</> SQL

```
START TRANSACTION;
```

```
UPDATE student  
SET marks=100  
WHERE id=1;
```

```
ROLLBACK;
```


`</>` SQL

```
START TRANSACTION;
```

```
UPDATE student  
SET marks=90  
WHERE id=1;
```

```
SAVEPOINT sp1;
```

```
UPDATE student  
SET marks=80  
WHERE id=2;
```

```
SAVEPOINT sp2;
```

```
UPDATE student  
SET marks=70  
WHERE id=3;
```

`</>` SQL

```
START TRANSACTION;
```

```
UPDATE student  
SET marks=90  
WHERE id=1;
```

```
SAVEPOINT sp1;
```

```
UPDATE student  
SET marks=80  
WHERE id=2;
```

```
UPDATE student  
SET marks=70  
WHERE id=3;
```

```
ROLLBACK TO sp1;
```

```
COMMIT;
```

7. AUTOCOMMIT

Check

SQL

```
SELECT @@autocommit;
```

Disable

SQL

```
SET autocommit=0;
```

Enable

SQL

```
SET autocommit=1;
```

</> SQL

```
SET autocommit=0;
```

```
START TRANSACTION;
```

```
UPDATE student  
SET marks=90  
WHERE id=1;
```

```
SAVEPOINT A;
```

```
UPDATE student  
SET marks=80  
WHERE id=2;
```

```
SAVEPOINT B;
```

```
UPDATE student  
SET marks=70  
WHERE id=3;
```

```
ROLLBACK TO B;
```

Bank Transfer

SQL

```
START TRANSACTION;
```

```
UPDATE account  
SET balance=balance-1000  
WHERE acc_no=101;
```

```
UPDATE account  
SET balance=balance+1000  
WHERE acc_no=102;
```

```
COMMIT;
```

Failure

SQL

```
START TRANSACTION;
```

```
UPDATE account
SET balance=balance-1000
WHERE acc_no=101;
```

-- Error

```
ROLLBACK;
```

Command

Purpose

START TRANSACTION

Transaction start karta hai

COMMIT

Changes permanently save karta hai

ROLLBACK

Changes undo karta hai

SAVEPOINT

Checkpoint banata hai

ROLLBACK TO SAVEPOINT

Savepoint tak rollback karta hai

RELEASE SAVEPOINT

Savepoint hata deta hai

AUTOCOMMIT

Automatic commit on/off